READING SHEET

Capstone Review & Assessment

Module 3 · Frontend: React & Next.js · Day 50 · Week 10

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Assessment day**	**The full session**

Tip How to use this sheet

The last sheet of Module 3. Part 1 is how today runs and what the assessors are watching. Part 2 is the peer review method. Part 3 is the assessment scope, including the ten concepts most often missed — read that list before the exam. Part 4 closes the module and sets up the backend track. Do not change your application today.

Part 1 · How Today Runs

TIME	WHAT HAPPENS
0:00–0:15	Setup — URLs open, repositories ready
0:15–1:45	Capstone presentations, eight minutes each
1:45–2:00	Break
2:00–2:45	Peer code review, in pairs
2:45–3:45	Module assessment
3:45–4:00	Results process, and what comes next

Warning No changes today

The version you verified yesterday is the version you present. Editing code between now and your slot is how a working application stops working. Set up before you sit down: live URL open, repository open, five files in tabs, the documents to hand.

The eight minutes

MINUTES	WHAT TO COVER
0–1	What it is and who it is for — no code
1–3	The main flow, live, from the deployed URL
3–5	One technical decision, and why you made it
5–6	The three attacks, refused in front of the room
6–7	PERF.md — before, after, and the cause
7–8	What you cut, and what you would do next

Present from the **deployed URL**, not localhost — it is what you were assessed on, and the only place the previews, environment variables and middleware are all real. Time is enforced: running long means the last two rows never happen, and those are the two that separate a demonstration from a defence.

What the assessors are looking for

CRITERION	WHAT THEY WATCH	WEIGHT
Functionality	Does the main flow complete, live, first time	25%
Architecture	Can you justify where each piece sits	20%
Auth and data	Do all three attacks fail, and do you know why	20%
Performance, access	The numbers, and a keyboard pass	15%
Deployment, docs	A URL that opens, a README that works	10%
Presentation	Clarity, honesty, and answers to questions	10%

They are marking **understanding, not perfection**. A modest application whose every decision you can explain outscores a larger one you cannot — the purpose of a capstone is to show what you know, and knowing why you did something is the part that transfers to the next project.

If something goes wrong

PROBLEM	WHAT TO DO
The live site is down	Roll back, and say so calmly
A feature fails on stage	Explain what should happen, move on
You run out of time	Skip to the attacks and the numbers
A question defeats you	Say so, and say how you would find out
The network fails	Present from the local build, and explain

Something goes wrong in most demonstrations, including professional ones, and composure is itself assessed — handling it calmly reads far better than pretending it did not happen. A short screen recording of the main flow, made yesterday, costs nothing and rescues a failed network.

Part 2 · Peer Code Review

Forty-five minutes reading somebody else's code and being read in return. It is a graded professional skill in its own right, and reading other people's work is how most developers actually improve.

#	IN THIS ORDER
1	Read the README, then open the live URL
2	Use the application before reading any code
3	Look at the route structure and the layouts
4	Follow one feature from URL to data and back

#	IN THIS ORDER
5	Check one protected action for both its checks
6	Write three strengths and three questions

Code makes sense only once you know what it is for, so reviewers who open the repository first spend the session guessing at intent. And follow *one* thread: forty-five minutes buys one feature understood properly, whereas skimming the whole repository produces comments nobody can act on.

What to look for

AREA	ASK	A GOOD SIGN
Boundary	Why is this a client component?	Client code only where it must be
Security	What stops me calling this action?	A check inside the write itself
Queries	Where does this id come from?	The session, not the request
States	What does a new account see?	Written empty and error states
Markup	Is this a button or a div?	Real elements doing real jobs
Naming	Could I find this file blind?	Names that describe behaviour

Every one of those is answerable by reading a single feature end to end, and every one maps to something assessed. They are also the questions a reviewer will ask you on your first job — which is the real reason this session exists.

Giving and receiving feedback

UNHELPFUL	HELPFUL
"This is wrong"	"What happens if this is null?"
"Bad naming"	"I expected getOrders to return orders"
"You should use X"	"X would handle the retry — worth it?"
Rewriting it for them	Naming the problem, not the fix
Only faults	Three strengths, then three questions

> **Tip Questions, not verdicts**
> A question invites an explanation, and half the time the explanation is good and you were wrong. A verdict invites defensiveness and teaches nobody anything. When receiving: if a reviewer misread something, the next person will too — confusion is data whether or not the code was correct. Note everything in FUTURE.md rather than fixing it today; acting on review during an assessment is how a passing project breaks.

Part 3 · The Module Assessment

One hour, written and practical, covering the whole of Modules 2 and 3. It tests whether you can **reason about the decisions** — not whether you memorised an API.

AREA	TOPICS
React core	Components, props, state, hooks, keys, effects
Routing and forms	Router, context, controlled inputs, validation
Next.js	Rendering strategies, the boundary, route handlers
Production	Auth, data fetching, performance, metadata
Judgement	Given a scenario, choose and justify an approach

The last row is the point. Questions are written as situations rather than definitions: *this screen behaves like this, what would you do and why*. If you can explain your own capstone honestly, you can answer most of them — which is why yesterday's documents were worth writing. Show your reasoning: on scenario questions the reasoning carries most of the marks, and there is no penalty for a wrong answer, so answer everything.

The ten things most often missed

CONCEPT	THE THING PEOPLE GET WRONG
Keys in a list	The index is not a stable id
State updates	They batch — use the functional form
useEffect cleanup	Every subscription and timer must be closed
The server boundary	A client component cannot be async
Rendering strategy	Reading cookies makes a route dynamic
Server actions	Middleware is not the security check
Scoped queries	Never trust an id from the request
NEXT_PUBLIC	Inlined at build, readable by anyone
metadataBase	Without it, previews lose their image
Forms	onSubmit on the form, not onClick

Marks and what follows

COMPONENT	WEIGHT
Final examination	50%
Homework, exercises and the capstone	35%
Attendance and participation	15%
Pass mark for the module	70%

Below 70% is not a failure — it triggers a structured remedial plan: targeted work on the specific gaps, then a reassessment. Nobody advances with an unresolved gap and nobody is left behind for having one. Marks and written feedback follow within the week, per criterion, with the specific things to work on rather than a single number.

Part 4 · Closing Module 3

YOU CAN NOW	EVIDENCE
Build a UI from composed, stateful components	Every screen of the capstone
Fetch on the server, cache, and revalidate	DATA.md
Layer security checks and scope every query	AUTH.md, three failed attacks
Measure, diagnose and improve performance	PERF.md
Ship to a live URL and roll back	The URL in your README

That list, with a public repository demonstrating each line, is what a hiring manager for a junior React position is looking for — and most applicants for those roles cannot show the last three rows.

What to keep doing

HABIT	WHY IT MATTERS
Push every day	The profile is the portfolio
Deploy everything you build	A URL beats a description
Read one other codebase a week	Faster than writing your own
Measure before optimising	The habit that separates seniors
Ask what happens without the UI	The security instinct, in one question

Your capstone is not finished — it is the first thing anyone will look at. Spend an evening on FUTURE.md over the coming weeks: a project that visibly improved after assessment says more than one that stopped. And keep the README current; it is the first file a reviewer opens and the one most likely to be out of date.

Module 4 — the backend track

TRACK	SUITS YOU IF	WHERE IT LEADS
Node.js with Nest.js	You want one language across the stack	The common startup stack
Golang	You liked reasoning about performance	Infrastructure, high throughput
.NET with C#	You prefer structure and strong typing	Enterprise and finance

Everyone shares Day 51: servers, HTTP, requests and responses, status codes and what an API is are taught to the whole cohort before the split. All three tracks cover the same core — API design, authentication and cloud deployment — so choose on evidence, and ask about the market for each in Addis rather than picking what sounds most interesting.

Everything you deployed this module talks to something. In Module 4 you build the API Addis Eats has been pretending to have; in Module 5, the three databases behind it; then testing, containers, pipelines and the architecture to justify it all. The same instincts apply throughout: decide deliberately, verify with tools, and never trust what the browser sends you.

Assigned Reading

Required

1. **Google — How to do a code review** What / why: the standard reference — what to look for, and in what order.
2. **MDN — An overview of HTTP** What / why: requests, responses and status codes — preparation for Day 51.

Optional — go deeper

- **Conventional comments** — a small convention that makes review feedback markedly easier to act on.
- **GitHub — Profile READMEs** — pinned repositories and making your work findable.

Today's Sequence

1. Set up before the session begins — URL, repository, five files, documents.
2. Present the eight minutes, and answer questions honestly rather than confidently.
3. Review your partner by using the application before opening any code.
4. Follow one feature end to end and check one protected action for both checks.
5. Write three specific strengths and three questions worth answering.
6. Sit the assessment, showing your reasoning on every scenario question.
7. Add everything today surfaced to FUTURE.md, and choose your backend track.

Closing Checklist

> **Tip Before you leave**
> Capstone presented and questions answered. Peer review written and handed over. Assessment complete. FUTURE.md updated with everything from today. Backend track chosen. Ten weeks, three modules, a deployed application and a public portfolio — Module 4 starts on Day 51.

Check yourself

- What is the one decision in your capstone you are most confident defending?
- What did a peer's code do better than yours, and will you adopt it?
- Which of the ten commonly missed concepts did you get wrong today?
- Which backend track are you choosing, and what evidence led you there?
- What is the first item in your FUTURE.md, and when will you do it?